

MODULE 38 · DOMAIN 4 - ACCELERATE WORKLOAD MIGRATION AND MODERNIZATION

Modernisation During Migration

Patterns outlive products. This module is deliberately pattern-first.

DOMAIN 4 of 4 EXAM WEIGHT 20% MODULE 38 of 44

An honest note on the tooling

- ◆ On 7 November 2025, Refactor Spaces, App2Container, Porting Assistant, .NET Refactoring and Microservice Extractor all closed to new customers.
- ◆ **Mainframe Modernization's managed runtime closed too.** The self-managed runtime remains, for Rocket Software and Blu Age.
- ◆ All point at **AWS Transform**, the agentic modernisation service. It is not on the exam's in-scope list.
- ◆ **So learn the patterns.** Strangler fig, containerise in place, replatform or refactor. Those are what questions test.

ON THE EXAM

When an entire tooling layer moves at once, the durable knowledge is the pattern each product implemented.

Migrate then modernise, or modernise then migrate. **Doing both at once doubles the risk and the unknowns.**

The strangler fig pattern

- ◆ **Put a routing layer in front of the monolith**, then extract one capability at a time behind it.
- ◆ **Each extraction is independently reversible**. Route back to the monolith and nothing else changes.
- ◆ The monolith shrinks gradually until what remains is small enough to retire or leave alone.
- ◆ It needs the routing layer to be reliable, and it takes longer than a rewrite would if a rewrite ever finished.

ON THE EXAM

The value is that every step ships. A big-bang rewrite delivers nothing until it delivers everything, which is why so many never do.

Containerising in flight

- ◆ **Containerising is a replatform, not a refactor.** The application is unchanged; only its packaging differs.
- ◆ It genuinely helps with density, consistent deployment, and a path towards further change later.
- ◆ **A containerised monolith is still a monolith.** It does not become scalable or independently deployable by being packaged.
- ◆ Tooling existed to automate this and has largely closed. The pattern is unaffected, and can be done by hand.

ON THE EXAM

Containerising buys deployment consistency and density. It does not buy the things people expect from microservices.

Mainframe: replatform or refactor?

REPLATFORM

Keep the code, change the runtime

- COBOL keeps running, on AWS
- Rocket Software runtime
- Faster, lower risk, exits the hardware
- You still employ COBOL developers

REFACTOR

Convert the code itself

- COBOL and PL/I converted to Java
- AWS Blu Age tooling
- Slower, and removes the skills problem
- Output needs review and testing

THE TIEBREAKER

The deciding question is usually people, not technology. If nobody can maintain COBOL in five years, replatforming defers that.

When not to modernise

- ◆ **The application is being retired.** Modernising something with eighteen months left is spending on a sunk asset.
- ◆ **A hard deadline dominates.** Modernisation is unbounded work; a lease expiry is not.
- ◆ **The application is commodity.** Repurchase it rather than modernise something that gives you no advantage.
- ◆ **Nobody understands it.** Modernising an application no one can explain means rediscovering its behaviour by breaking it.

ON THE EXAM

Modernisation earns its place when the current architecture is genuinely the constraint. Otherwise it is expensive tidying.

Reading the scenario

THE SCENARIO SAYS

Extract services incrementally, reversibly

Hard deadline, and leadership wants microservices

COBOL, and nobody left who writes it

Exit the mainframe hardware quickly

Application retires in eighteen months

Containerise the monolith to make it scale

IT IS POINTING AT

Strangler fig with a routing layer

Migrate first, modernise after

Refactor to Java, not replatform

Replatform, keeping the COBOL

Do not modernise it

Correct the expectation first

ON THE EXAM

Ask what the modernisation is for. If the answer is because we are moving anyway, that is not a reason.

Meridian Freight's order monolith must eventually become services, but it also must leave the data centre in seven months. Leadership wants both, arguing that migrating twice is wasteful. The application team has never built a microservice.

The monolith works and is well understood.

What should the solutions architect recommend?

- ☐ A Rehost to meet the deadline, then apply the strangler fig pattern to extract services incrementally in AWS
- ☐ B Decompose into microservices as part of the migration, to avoid migrating twice
- ☐ C Containerise the monolith during migration, which delivers the microservice benefits
- ☐ D Delay the migration until the decomposition is complete

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

Rehost, then strangler fig
The deadline is fixed and decomposition is not. Rehosting meets it, and extraction afterwards happens without a lease expiring behind it.
- B

Decompose during migration
Two simultaneous changes, an inexperienced team and a fixed deadline. When something breaks, nobody can tell which change caused it.
- C

Containerise for the benefits
A containerised monolith is still a monolith. It changes packaging, not architecture, and the expectation is wrong.
- D

Delay the migration
The data centre exit is not negotiable. Delaying it for an unbounded decomposition risks having nowhere to run.

THE SIGNAL

Migrating twice is not wasteful; it is sequencing. Leadership's argument sounds efficient and ignores that the deadline is fixed.

A retailer wants to extract the pricing engine from a large monolith already running on AWS. The monolith must keep serving all other traffic throughout, and any extraction must be reversible within minutes if pricing behaviour differs.

There is no deadline pressure.

What should the solutions architect recommend?

- ☐ A A strangler fig approach: a routing layer in front, pricing requests routed to a new service, reversible by changing the route
- ☐ B Rewrite the monolith as microservices in a single release
- ☐ C Duplicate the monolith and remove everything except pricing
- ☐ D Leave the monolith intact and scale it vertically

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

Strangler fig with routing
One capability moves, everything else is untouched, and reversal is a routing change measured in minutes. Every requirement is met directly.
- B

Single-release rewrite
The opposite of reversible. A big-bang rewrite cannot be undone in minutes, and it changes everything at once.
- C

Duplicate and strip
Produces a second monolith carrying all the original coupling. It is a copy, not an extraction.
- D

Scale vertically
Declines to extract anything. It may be reasonable elsewhere, and it answers none of the stated requirements.

THE SIGNAL

Reversible within minutes is the phrase. Only a routing change is that fast, which is precisely what the pattern provides.

A bank runs core account processing on a mainframe in COBOL. Its four remaining COBOL developers retire within three years and the skill is not replaceable in its market. The business accepts a multi-year programme and has funded it.

The mainframe hardware contract has four years to run.

What should the solutions architect recommend?

- A Refactor: convert the COBOL to Java with automated tooling, then review, test and operate it with available skills
- B Replatform: keep the COBOL and run it on a managed runtime on AWS
- C Retain the mainframe and recruit COBOL developers
- D Repurchase a commercial core banking platform

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Refactor to Java
The constraint is skills, not hardware. Only refactoring removes the COBOL dependency, and the funding and timescale to do it exist.
- B** Replatform
Exits the hardware and keeps the COBOL, so in three years the code runs on AWS and nobody can maintain it.
- C** Recruit COBOL developers
The scenario says the skill is not replaceable in its market. Recruiting into a shrinking pool defers the problem at rising cost.
- D** Repurchase
Core account processing is a bank's most differentiated and regulated system. It is the least suitable candidate for a package.

THE SIGNAL

Read which constraint has the deadline. The hardware has four years; the people have three, and only one option addresses people.

During migration planning, a team proposes containerising a Java monolith with App2Container, expecting it to become horizontally scalable and independently deployable. The monolith holds session state in memory and writes to a local filesystem. The migration deadline is five months.

What should the solutions architect advise?

- A Correct the expectation: containerising changes packaging, not architecture, and the state assumptions must be fixed first
- B Proceed; containerising will make it horizontally scalable
- C Containerise and run a single replica to avoid the state problem
- D Refactor the monolith into microservices within the five months

◆ PAUSE AND DECIDE

What should the solutions architect advise?

- A** Correct the expectation
In-memory session state and local filesystem writes prevent horizontal scaling regardless of packaging. Externalise state first, or accept it does not scale.
- B** It will scale
A second replica loses sessions and diverges on local files. Packaging does not remove state assumptions.
- C** Single replica
Honest about the constraint and delivers none of the benefits claimed. It is a container running a monolith on one node.
- D** Refactor in five months
Overcorrects. A full decomposition inside a migration deadline is exactly what module 34 warns against.

THE SIGNAL

The state details are planted deliberately. In-memory sessions and local file writes are what actually block scaling.

A migration portfolio includes a claims processing application scheduled for replacement by a purchased platform in eighteen months. The migration team has scoped nine months of work to containerise it and move it to ECS during the data centre exit.

A simple rehost would take three weeks.

What should the solutions architect recommend?

- ☐ A Rehost it in three weeks; do not modernise an application being retired in eighteen months
- ☐ B Containerise as scoped, since it will run better on ECS
- ☐ C Retain it on-premises until the replacement arrives
- ☐ D Accelerate the replacement so the application never migrates

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Rehost in three weeks
Nine months of modernisation on something with eighteen months to live, then thrown away. Rehosting spends three weeks and meets the exit.
- B** Containerise as scoped
Nine months of engineering with roughly nine months of use afterwards. The investment never returns.
- C** Retain on-premises
The data centre is closing. Retain needs somewhere to retain it, which is the same objection as module 34.
- D** Accelerate the replacement
Changing another programme's timeline to avoid three weeks of rehosting work. It is not an architectural recommendation.

THE SIGNAL

Eighteen months against nine months of work is the arithmetic. Modernisation must have enough remaining life to repay it.

A company must exit its mainframe within twenty months because the hardware lease ends and will not be renewed. It employs a stable team of COBOL developers who are not retiring. The application is well documented and works.

A consultant has proposed a full refactor to Java.

What should the solutions architect advise?

- ☐ A Replatform: run the existing COBOL on a managed runtime on AWS, meeting the deadline and keeping the team productive
- ☐ B Refactor to Java as proposed
- ☐ C Renegotiate the hardware lease
- ☐ D Repurchase a packaged equivalent

◆ PAUSE AND DECIDE

What should the solutions architect advise?

- A

Replatform, keep the COBOL
The constraint is hardware, not skills. The team is stable, the code works, and replatforming exits the mainframe well inside twenty months.
- B

Refactor to Java
Solves a skills problem this company does not have, on a fixed deadline, and produces Java the COBOL team must then learn to maintain.
- C

Renegotiate the lease
The scenario says it will not be renewed. Changing a stated constraint is not an architectural answer.
- D

Repurchase
Replacing a working, well-documented core system inside twenty months, with all the process change that implies.

THE SIGNAL

Pair this with scenario three. Same two options, opposite answers, because the binding constraint changed from people to hardware.

What to carry out of this module

- ◆ **Migrate then modernise** unless the change is small and bounded. Doing both at once doubles the unknowns.
- ◆ **Strangler fig ships every step** and reverses with a routing change. A big-bang rewrite delivers nothing until it delivers everything.
- ◆ **A containerised monolith is still a monolith.** In-memory state and local files block scaling regardless of packaging.
- ◆ **Mainframe: replatform keeps the code, refactor keeps the skills.** Decide by which constraint binds first.

ON THE EXAM

The modernisation tooling layer largely closed in November 2025. Patterns are what the exam tests, and what survives a rename.